



ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ BMS Institute of Technology and Management

(An Autonomous Institution Affiliated to VTU, Belagavi)
Avalahalli, Doddaballapur Main Road, Bengaluru – 560119



B E – Computer Science and Engineering

Internal Assessment – II

TERM: 2025 – 2026 Even Semester		COURSE NAME: Compiler Design
DATE/TIME : 12-05-2026 TIME:9.30-11.00AM		COURSE CODE : BCS603
SEMESTER: VI Semester	MAX MARKS: 40	COURSE COORDINATORS: Dr. Mohammed Khurram Prof. Manjula S D, Prof. Chetan D S, Prof. A. Mari Kirthima

Max. Marks: 40

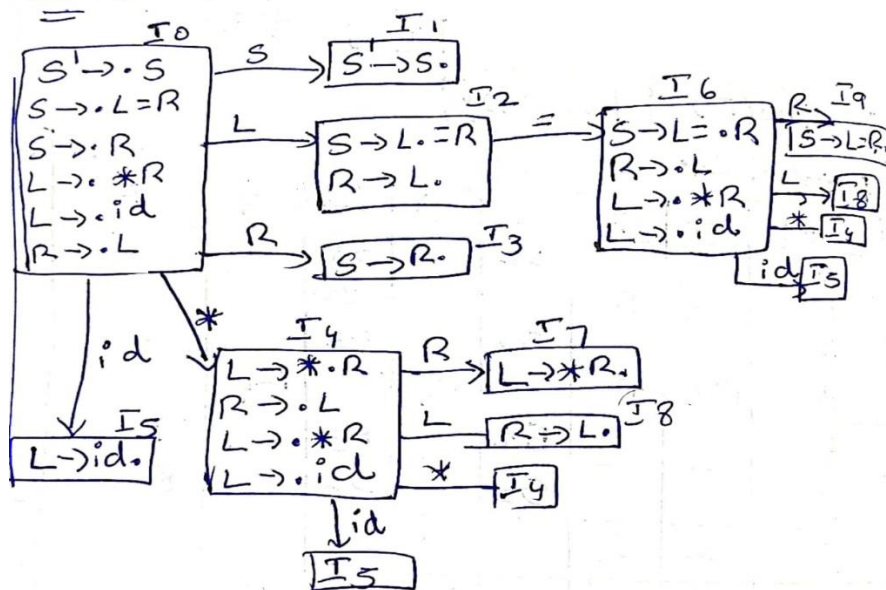
Q. NO	Questions	Marks																																										
1.	Apply the concept of handle pruning to demonstrate the working of a shift-reduce parser for accepting the input string int id, id; and draw its parse tree using the given grammar: S → TL; T → int float L → L, id id <table border="1" style="margin-right: 20px;"> <thead> <tr> <th>Stack</th><th>Input buffer</th><th>Action</th></tr> </thead> <tbody> <tr><td>\$</td><td>int id, id, id, \$</td><td>Shift</td></tr> <tr><td>\$int</td><td>id, id, id, \$</td><td>Reduce T → int</td></tr> <tr><td>\$T</td><td>id, id, id, \$</td><td>Shift</td></tr> <tr><td>\$T id</td><td>, id, id, \$</td><td>Reduce L → L, id</td></tr> <tr><td>\$TL</td><td>, id, id, \$</td><td>Shift</td></tr> <tr><td>\$TL,</td><td>id, id, \$</td><td>Shift</td></tr> <tr><td>\$TL, id</td><td>, id, \$</td><td>Reduce L → L, id</td></tr> <tr><td>\$TL</td><td>, id, \$</td><td>Shift</td></tr> <tr><td>\$TL,</td><td>id, \$</td><td>Shift</td></tr> <tr><td>\$TL, id</td><td>, \$</td><td>Shift</td></tr> <tr><td>\$TL</td><td>, \$</td><td>Shift</td></tr> <tr><td>\$TL,</td><td>\$</td><td>Shift</td></tr> <tr><td>\$TL;</td><td>\$</td><td>Accept</td></tr> </tbody> </table> <u>Parse tree</u> <pre> graph TD S --> T1[T] S --> L1[L] S --> C1[,] S --> L2[L] S --> Semicolon[;] T1 --> int[int] L1 --> id1[id] L2 --> id2[id] </pre>	Stack	Input buffer	Action	\$	int id, id, id, \$	Shift	\$int	id, id, id, \$	Reduce T → int	\$T	id, id, id, \$	Shift	\$T id	, id, id, \$	Reduce L → L, id	\$TL	, id, id, \$	Shift	\$TL,	id, id, \$	Shift	\$TL, id	, id, \$	Reduce L → L, id	\$TL	, id, \$	Shift	\$TL,	id, \$	Shift	\$TL, id	, \$	Shift	\$TL	, \$	Shift	\$TL,	\$	Shift	\$TL;	\$	Accept	10 (6M for handle pruning+4M for parse tree)
Stack	Input buffer	Action																																										
\$	int id, id, id, \$	Shift																																										
\$int	id, id, id, \$	Reduce T → int																																										
\$T	id, id, id, \$	Shift																																										
\$T id	, id, id, \$	Reduce L → L, id																																										
\$TL	, id, id, \$	Shift																																										
\$TL,	id, id, \$	Shift																																										
\$TL, id	, id, \$	Reduce L → L, id																																										
\$TL	, id, \$	Shift																																										
\$TL,	id, \$	Shift																																										
\$TL, id	, \$	Shift																																										
\$TL	, \$	Shift																																										
\$TL,	\$	Shift																																										
\$TL;	\$	Accept																																										
	OR																																											
2.	Consider the following Grammar: S → L = R S → R L → * R L → id R → L Use SLR parsing technique , Construct SLR parsing table and																																											

- Identify conflicts
- Why does SLR fail here?
- Which method will work?

Augmented Grammar

$S' \rightarrow S \rightarrow 0$
 $S \rightarrow L = R \rightarrow 1$
 $S \rightarrow R \rightarrow 2$
 $L \rightarrow * R \rightarrow 3$
 $L \rightarrow id \rightarrow 4$
 $R \rightarrow L \rightarrow 5$

LR(0) items with Goto operation



compute Follow(S) = { \$ }
 Follow(L) = { =, \$ }
 Follow(R) = { \$ }

SLR(1) parsing Table

states	Action				Goto		
	id	=	*	\$	S	L	R
0	S5		S4		1	2	3
1			Accept				
2		S6 S5		r5			
3				r2			
4	S5		S4			8	7
5		r4		r4			
6	S5		S4			8	9
7		r3		r3			
8		r5		r5			
9				r1			

- In State 2, there is a Shift-Reduce Conflict on the input symbol '='

10

(1M

Augmented

Grammar+4 M

for LR(0) items

+2 M for table+

1 M for each

sub question



ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ BMS Institute of Technology and Management

(An Autonomous Institution Affiliated to VTU, Belagavi)
Avalahalli, Doddaballapur Main Road, Bengaluru - 560119



- (b) SLR fails because it uses the simple FOLLOW set to determine when to reduce. Here, $(R \rightarrow L)$ is reduced if the next symbol is in $\text{FOLLOW}(R)$. Since '=' is in $\text{FOLLOW}(R)$ and also follows L in the $S \rightarrow L=R$ production, the parser cannot decide whether to shift the '=' (trying to match $S \rightarrow L=R$) or reduce L to R (trying to match $S \rightarrow R$ with $R \rightarrow L$)
- (c) The Canonical LR(1) (CLR) parsing method will work. CLR uses lookahead items specifically associated with each reduction, allowing it to distinguish that $R \rightarrow L$ should not be reduced when '=' is the next symbol. LALR(1) will also work as it resolves this specific conflict

3 Construct a Goto graph and CLR parse table for the following grammar

$S \rightarrow aAd \mid bBd \mid aBe \mid bAe$

$A \rightarrow c$

$B \rightarrow c$

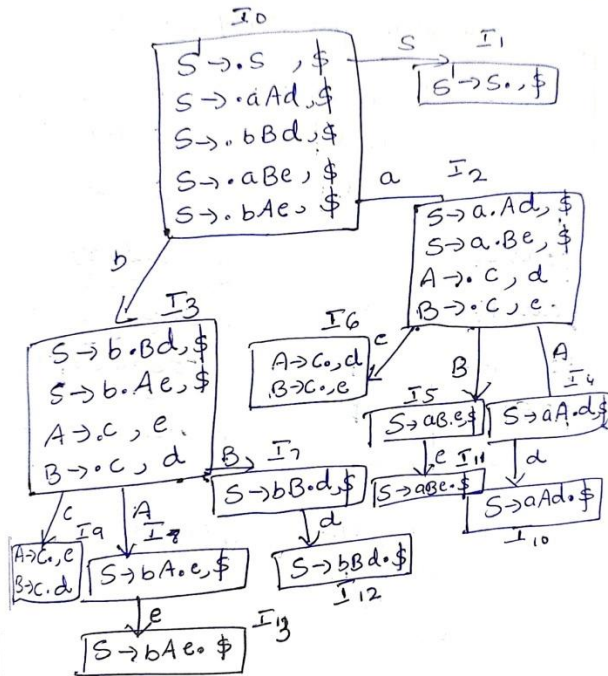
Augmented grammar

$S' \rightarrow S \rightarrow 0$
 $S \rightarrow aAd \rightarrow 1$
 $S \rightarrow bBd \rightarrow 2$
 $S \rightarrow aBe \rightarrow 3$
 $S \rightarrow bAe \rightarrow 4$
 $A \rightarrow c \rightarrow 5$
 $B \rightarrow c \rightarrow 6$

$\text{First}(S) = \{a, b\}$

$\text{First}(A) = c$

$\text{First}(B) = c$



paying Table

states	a	b	c	d	e	\$	S	A	B
0	S2	S3					1		
1						Accept			
2			S6					4	5
3			S9					8	7
4				S10					
5					S11				
6				S5	S6				
7				S12					
8					S13				
9				S6	S5				
10						S1			
11						S3			
12						S2			
13						S4			

10

(1M

Augmented
Grammar+5 M

for Goto

Graph+4M

table)

	OR																																																	
4	Given the following grammar: $S \rightarrow AA$ $A \rightarrow aA \mid b$ Construct a set of LR(0) items and LR Parsing table <u>Step1:</u> Augmented grammar is $\begin{array}{ll} S' \rightarrow S & \rightarrow 0 \\ S \rightarrow AA & \rightarrow 1 \\ A \rightarrow aA & \rightarrow 2 \\ A \rightarrow b & \rightarrow 3 \end{array}$ <u>Step2:</u> Canonical collection of LR(0) items DFA <u>Step3:</u> LR(0) parse Table <table><thead><tr><th></th><th>a</th><th>b</th><th>\$</th><th>S</th><th>A</th></tr></thead><tbody><tr><td>0</td><td>S3</td><td>S4</td><td></td><td>1</td><td>2</td></tr><tr><td>1</td><td></td><td></td><td>Accept</td><td></td><td></td></tr><tr><td>2</td><td>S3</td><td>S4</td><td></td><td></td><td>5</td></tr><tr><td>3</td><td>S3</td><td>S4</td><td></td><td></td><td>6</td></tr><tr><td>4</td><td>r3</td><td>r3</td><td>r3</td><td></td><td></td></tr><tr><td>5</td><td>r1</td><td>r1</td><td>r1</td><td></td><td></td></tr><tr><td>6</td><td>r2</td><td>r2</td><td>r2</td><td></td><td></td></tr></tbody></table>		a	b	\$	S	A	0	S3	S4		1	2	1			Accept			2	S3	S4			5	3	S3	S4			6	4	r3	r3	r3			5	r1	r1	r1			6	r2	r2	r2			10(1 M Augmented Grammar+5 M for LR(0) items +4M for table)
	a	b	\$	S	A																																													
0	S3	S4		1	2																																													
1			Accept																																															
2	S3	S4			5																																													
3	S3	S4			6																																													
4	r3	r3	r3																																															
5	r1	r1	r1																																															
6	r2	r2	r2																																															
5.	A company developing a smart billing calculator wants to design a compiler component that evaluates multiplication expressions during parsing using a top-down parsing technique. The calculator supports only single-digit operands and the multiplication operator. Given the grammar $T \rightarrow T * F \mid F$ and $F \rightarrow \text{digit}$, redesign the grammar suitable for top-down parsing, Analyze the SDD with appropriate semantic rules to evaluate expressions, and demonstrate the evaluation process by drawing the annotated parse tree and dependency graph for the input string $3 * 4$. Solution : The given grammar has left recursion and hence it is not suitable for top-down parser. To make it suitable for top-down parsing, we have to eliminate left recursion. After eliminating left recursion , the following grammar is obtained: $\begin{array}{l} T \rightarrow F T' \\ T' \rightarrow * F T' \mid \epsilon \\ F \rightarrow \text{digit} \end{array}$ Annotated Parse tree and Dependency Graph for $3 * 4$ Productions $T \rightarrow F T'$ $T' \rightarrow * F T'_1$ $T' \rightarrow \epsilon$ Semantic Rules $T'.inh = F.val$ $T.val = T'.syn$ $T'_1.inh = F.val \times T'.inh$ $T'.syn = T'_1.syn$ $T'.syn = T'.inh$	10(2M for eliminating recursion+4 M for semantic rules+4 M for annotated parse tree &dependency graph)																																																



ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ BMS Institute of Technology and Management

(An Autonomous Institution Affiliated to VTU, Belagavi)
Avalahalli, Doddaballapur Main Road, Bengaluru - 560119



OR

6.

A team is developing a compiler for a simple expression evaluation language used in a calculator application. The compiler must analyze arithmetic expressions and internally represent them as syntax trees so that later phases can process them efficiently. The grammar of the language is defined as

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow (E) \mid \text{digit} \mid \text{id}$

(a) Analyze the Syntax Directed Definition (SDD) with appropriate semantic rules to construct a syntax tree for the given grammar.

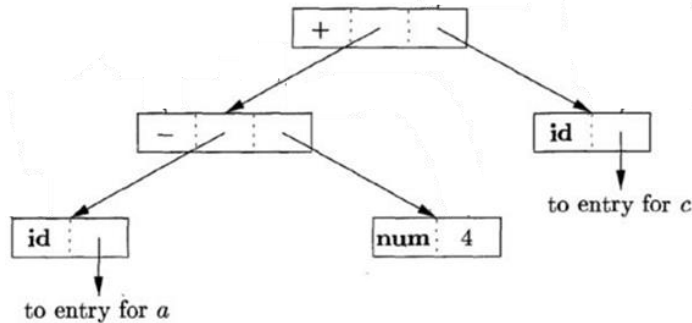
(b) Using your semantic rules, construct and draw the syntax tree for the expression: $a-4+c$

Solution:

PRODUCTION	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	$E.\text{node} = \text{new Node}('+', E_1.\text{node}, T.\text{node})$
2) $E \rightarrow E_1 - T$	$E.\text{node} = \text{new Node}('-', E_1.\text{node}, T.\text{node})$
3) $E \rightarrow T$	$E.\text{node} = T.\text{node}$
4) $T \rightarrow (E)$	$T.\text{node} = E.\text{node}$
5) $T \rightarrow \text{id}$	$T.\text{node} = \text{new Leaf}(\text{id}, \text{id.entry})$
6) $T \rightarrow \text{num}$	$T.\text{node} = \text{new Leaf}(\text{num}, \text{num.val})$

10(6M for
semantic rules
+4 M for
syntax tree)

Syntax tree for the expression: $a-4+c$



7.a

Consider the following conditional statement:

if $(a < b)$

$t = 1;$

else

$t = 2;$

Apply backpatching technique to translate the if-else statement into intermediate code and explain how backpatching ensures correct execution order.

Solution:

100: if $a < b$ goto 102

101: goto 104

102: $t = 1$

103: goto 105

104: $t = 2$

105:

5(3M for IR+2
M for
Explanation of
backpatching
process)

	Backpatching Process: - backpatch(100.truelist, 102): Patches line 100 to jump to t=1 (line 102) if a < b is true. - backpatch(100.falselist, 104): Patches line 101 to jump to t=2 (line 104) if a < b is false. - backpatch(103.nextlist, 105): (If more code existed, skips over t=2 after t=1 executes)																																									
7.b	Consider the grammar: B -> B1 && B2 B -> B1 B2 B -> ! B1 Construct semantic rules for each production to generate intermediate code using backpatching. $B \rightarrow B_1 M B_2 \quad \{ \text{backpatch}(B_1.\text{falselist}, M.\text{instr});$$B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist});$$B.\text{falselist} = B_2.\text{falselist}; \}$$B \rightarrow B_1 \&\& M B_2 \quad \{ \text{backpatch}(B_1.\text{truelist}, M.\text{instr});$$B.\text{truelist} = B_2.\text{truelist};$$B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist}); \}$$B \rightarrow ! B_1 \quad \{ B.\text{truelist} = B_1.\text{falselist};$$B.\text{falselist} = B_1.\text{truelist}; \}$	5(2M for OR+2M for AND +1M NOT)																																								
	OR																																									
8.	Consider the arithmetic expression: x = a + b * c - b * c - Generate the three-address code (TAC) for the above expression. - Represent the TAC using: Quadruples, Triples and Indirect Triples. Identify any common subexpressions in the given expression and construct a DAG for the expression. Solution: 1.Three-Address Code (TAC) t1 = b * c; t2 = a + t1; t3 = t2 - t1; x = t3; 2.Representations of TAC using Quadruples Quadruples use four fields: Op, Arg1, Arg2, Result Triples avoid temporary variable names by referencing the position of previous operations. Quadrples <table><tr><th>op</th><th>arg1</th><th>arg2</th><th>result</th></tr><tr><td>*</td><td>b</td><td>c</td><td>t1</td></tr><tr><td>+</td><td>a</td><td>t1</td><td>t2</td></tr><tr><td>-</td><td>t2</td><td>t1</td><td>t3</td></tr><tr><td>=</td><td>t3</td><td></td><td>x</td></tr></table> Tiples <table><tr><th></th><th>op</th><th>arg1</th><th>arg2</th></tr><tr><td>(0)</td><td>*</td><td>b</td><td>c</td></tr><tr><td>(1)</td><td>+</td><td>a</td><td>(0)</td></tr><tr><td>(2)</td><td>-</td><td>(1)</td><td>(0)</td></tr><tr><td>(3)</td><td>=</td><td>x</td><td>(2)</td></tr></table>	op	arg1	arg2	result	*	b	c	t1	+	a	t1	t2	-	t2	t1	t3	=	t3		x		op	arg1	arg2	(0)	*	b	c	(1)	+	a	(0)	(2)	-	(1)	(0)	(3)	=	x	(2)	10(2M for TAC+2M for Quadrples+2M triple+2M indirect triple+2M DAG)
op	arg1	arg2	result																																							
*	b	c	t1																																							
+	a	t1	t2																																							
-	t2	t1	t3																																							
=	t3		x																																							
	op	arg1	arg2																																							
(0)	*	b	c																																							
(1)	+	a	(0)																																							
(2)	-	(1)	(0)																																							
(3)	=	x	(2)																																							



ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ BMS Institute of Technology and Management

(An Autonomous Institution Affiliated to VTU, Belagavi)
Avalahalli, Doddaballapur Main Road, Bengaluru – 560119

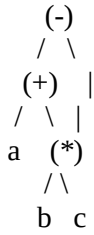


Indirect Triples

100	(0)
101	(1)
102	(2)
103	(3)

	op	arg1	arg2
(0)	*	b	c
(1)	+	a	(0)
(2)	-	(1)	(0)
(3)	=	x	(2)

DAG for the given expression is



Course Outcomes (COs)

CO1:	Apply compiler structure concepts to perform lexical analysis of source code.
CO2:	Apply the concept of tokenization, parsing, code generation, code optimization for the given piece of code written in any language.
CO3:	Analyze the syntax-directed translation techniques in the given grammar.
CO4:	Make use of suitable techniques to generate the intermediate code and target code.

Bloom's Category

Remembering (K1)	Understanding (K2)	Applying (K3)	Analyzing (K4)	Evaluating (K5)	Creating (K6)
------------------	--------------------	---------------	----------------	-----------------	---------------

Course Coordinator (s)	Module Coordinator (s)	Program Coordinator (s)	BoE Chairman